An Alternative Learning-Based Approach for Economic Dispatch in Smart Grid

Fanghong Guo, *Member, IEEE*, Bowen Xu, Lantao Xing, *Member, IEEE*, Wen-An Zhang, *Member, IEEE*, Changyun Wen, *Fellow, IEEE*, and Li Yu, *Senior Member, IEEE*

***Abstract*—This article tries to provide a new alternative approach to solve the economic dispatch (ED) problem in a smart grid system. Such a problem has been widely studied recently with several advanced numerical optimization algorithms being proposed. However, most of these numerical algorithms may suffer from high computational cost for on-line optimization. In this article, we aim to address this problem by proposing a learning-based optimization strategy. The key idea is to regard the optimization strategy of the ED problem as an unknown mapping relationship. With the help of traditional ED optimization algorithms to obtain the ground truth, we employ a deep neural network (DNN) to learn the ED optimization strategy and use it for online ED. In particular, our main contribution in this article is to theoretically show that one popular ED algorithm, i.e., λ-iteration algorithm, can be accurately approximated by a well-constructed DNN with finite network size. Moreover, dynamic units status of dispatchable generators is also considered and can be well solved by our proposed approach. Furthermore, several simulation case studies implemented on a 3-unit power system and an IEEE-30 bus power system validate the effectiveness of our proposed method.**

***Index Terms*—λ-iteration algorithm, deep neural network (DNN), economic dispatch (ED), learning to optimize, smart grid.**

I. Introduction

FOR DECADES, smart grid, as an enhancement of traditional power system, has attracted the attention of many researchers due to its efficiency, reliability, sustainability and economy [1]. Different from fossil fuel-driven generators, renewable energy sources play an important role in powering smart grids. In addition, it is equipped with more advanced measuring sensors, communication infrastructure and intelligent control technologies. Economic dispatch (ED), as one of the fundamental but challenging optimization problems in smart grid systems, aims to minimize the overall power generation cost, while subjecting to the generation-demand balance constraint and local generation constraints. In recent years, significant achievements have been made in ED optimization algorithm development. Various kinds of optimization algorithms have been proposed, including convex optimization approaches and heuristic search-based methods [2]–[10].

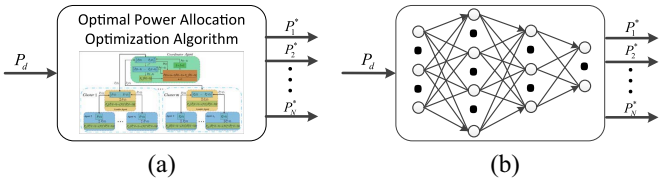

Fig. 1. Comparison illustration between the traditional optimization algorithm and learning-based algorithm. (a) Traditional optimization algorithm. (b) Learning-based algorithm.

Even though these algorithms have been proven to achieve excellent performance through theoretical analysis and numerical simulations, applying them for practical dispatch in smart grids still faces many challenging and tight situations. For example, most existing numerical algorithms, including heuristic algorithms [3] and gradient-based approaches [5], are designed to solve the ED problem in an iterative way, which is usually resource consuming and time consuming. Moreover, almost all heuristic algorithms contain a series of random strategies with no guarantees on the determinacy of the final solution [11]–[15]. Besides, the involved parameters of the optimization algorithm (e.g., the regularization parameters in the ADMM-based algorithm [8]) is difficult to determine in dynamic scenarios.

In order to address the above concerns, we may as well consider the ED problem from a system point of view. As depicted in Fig. 1(a), we readily know that all ED algorithms are set to do one thing, that is, for a given total load demand P_d, deducing and establishing detailed theoretical support to find the optimal solution P_i^*. Motivated by recent progress in machine learning technologies [16]–[25], we would like to pose a constructive question: If we regard the relationship between the input and output of an ED optimization problem as an unknown "black box," can we use a deep neural network (DNN) to learn it for online ED or even further potential applications? The answer should be positive. Going back to the 1990s, there have been several pioneering works [26], [27]

Manuscript received October 25, 2020; revised March 12, 2021; accepted April 9, 2021. Date of publication April 13, 2021; date of current version September 23, 2021. This work was supported in part by the National Natural Science Foundation of China under Grant 61903333 and Grant 61822311; in part by the Key Research and Development Program of Zhejiang Province under Grant 2019C03098 and Grant 2020C01109; in part by the Zhejiang Qianjiang Talent Project under Grant QJD1902010; and in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LQ19F030008. *(Fanghong Guo and Bowen Xu contributed equally to this work.) (Corresponding author: Wen-An Zhang.)*

Fanghong Guo, Bowen Xu, Wen-An Zhang, and Li Yu are with the Department of Automation, Zhejiang University of Technology, Hangzhou 310023, China (e-mail: fhguo@zjut.edu.cn; bwxu@zjut.edu.cn; wazhang@zjut.edu.cn; lyu@zjut.edu.cn).

Lantao Xing and Changyun Wen are with the School of Electrical and Electronics Engineering, Nanyang Technological University, Singapore (e-mail: lantao.xing@ntu.edu.sg; ecywen@ntu.edu.sg).

Digital Object Identifier 10.1109/JIOT.2021.3072840

employing artificial neural networks to solve the ED problem in power systems. However, the limited technological support and computational resources at that time disabled such forward methods to star and shine on efficiently solving an ED problem. Moreover, none of these works pay attention to the reason why neural networks can approximate the complicated algorithms they focus on. This contributes to the main motivation of this article.

In this work, we advocate to use a DNN to accurately learn the ED optimization strategy for a smart grid. Furthermore, we aim to set a basic example for answering that how many layers/neurons are needed to approximate a given "learnable" algorithm. Our original intention is that if such a learning task can be well completed, then we must be willing to use simple matrix multiplication calculations in DNN to replace complicated, time-consuming iterative operations to achieve real-time ED. In this scenario, all we need to do is feed a given load demand P_d and collect the optimal power output P_i^* through several simple matrix operations, as shown in Fig. 1(b). In addition, we should mention that the cost of training such a DNN is considerably acceptable since the data set can be obtained from the learned optimization algorithm, and techniques and optimizers to accelerate training have been widely developed in recent years [28].

Given that the ED problem can be formulated in different forms, (e.g., considering valve-point effect [29], multiple fuel types [30]) and the corresponding optimization algorithms are diverse in terms of rationale and complexity, it is unrealistic to prove that DNN can learn all available ED algorithms. In this article, for preliminary theoretical analysis, we select a general convex formulation for our ED problem, while the classic λ-iteration algorithm [31] is adopted to play the role of "the algorithm to be learned" due to its simplicity and efficiency with strong theoretical support [32]. By *deeply unfolding* the λ-iteration algorithm along its complete iterations, it is proved that the mapping relationship in each iteration of this popular algorithm can be represented by some basic units. Then for a given approximation accuracy, it can be revealed quantitatively that at most how many hidden layers and neurons are needed for constructing a "well-learned" DNN.

The main contributions of this article are summarized in the following.

1) We propose a new learning-based approach to solve the ED problem in real time.
2) In addition to satisfactory real-time performance and solution accuracy, the proposed approach shows excellent adaptability to dynamic units status in smart grid, which, to the best of our knowledge, has not been well solved by existing learning-based approaches.
3) We set a basic example for efficiently solving an ED problem with a learning-based proposition, which paves the way for real-time control tasks far beyond ED.
4) It is rigorously proved that at most how many layers and neurons are required for a DNN approximating a given iterative ED algorithm in [31].

To promote further study of our DNN-based approach for solving more problems far beyond ED, the code used to generate the simulation results in this article is available online: https://github.com/xbwwwx/DNN-ED-IoT.

The remainder of this article is organized as follows. In Section II, the ED problem is formally formulated, which is followed by a brief introduction of λ-iteration algorithm. Section III introduces the details of our proposed approach. Section IV provides the theoretical analysis of our proposed method. The simulation results supporting our algorithm and the corresponding theorem are shown in Section V. Finally, the conclusion is given in Section VI.

II. Economic Dispatch Problem in Smart Grid

A. Problem Formulation

ED, as one of the well-studied key problems in power system research, aims to find the optimum power allocation among a group of generators subject to the generation-demand balance constraint and local generation constraints. It can be considered as the following optimization problem:

$$\begin{cases} \min_{P_i} \sum_{i=1}^{N} f_i(P_i) \\ \text{s.t.} \quad \sum_{i=1}^{N} P_i = P_d \\ \qquad P_i^{\min} \le P_i \le P_i^{\max} \end{cases} \tag{1}$$

where N is the number of dispatchable generators (DGs), P_d denotes the total load demand, $P_i^{\max}$ and $P_i^{\min}$ indicate the maximum and minimum power outputs of the ith generator, respectively. The cost function of the ith generator, i.e., f_i, can be represented by a quadratic function as follows [10]:

$$f_i(P_i) = a_i P_i^2 + b_i P_i + c_i \tag{2}$$

with a_i, b_i, and c_i being the cost coefficients.

From mathematical point of view, the ED problem formulated in (1) is a standard convex optimization problem with both equality and inequality constraints. In fact, as reviewed in Section I, there are various kinds of algorithms to solve this problem. In the next section, one popular ED algorithm, i.e., λ-iteration optimization algorithm, will be introduced as a preliminary.

B. λ-Iteration Optimization Algorithm

First, the incremental cost of each generator is defined as

$$\lambda_i = \frac{\partial f_i(P_i)}{\partial P_i} = g_i(P_i), \quad i = 1, \ldots, N. \tag{3}$$

It is well known that if the incremental cost λ_i of all generators reach consensus to λ_i^*, then the optimal solution of each generator P_i^* can be derived as [31]

$$P_i^* = \begin{cases} g_i^{-1}(\lambda^*), & \text{if } P_i^{\min} \le g_i^{-1}(\lambda^*) \le P_i^{\max} \\ P_i^{\max}, & \text{if } g_i^{-1}(\lambda^*) > P_i^{\max} \\ P_i^{\min}, & \text{if } g_i^{-1}(\lambda^*) < P_i^{\min} \end{cases} \tag{4}$$

where $g_i^{-1}(\lambda)$ denotes the inverse function of $g_i(P_i)$ in (3).

There are several methods to find such an optimal incremental cost λ^*. Among them, λ-iteration algorithm is most popular due to its simple implementation and acceptable convergence speed [31]. The rationale behind this algorithm is to find the optimal incremental cost λ^* of all the generators by using the

Algorithm 1 Pseudocode of λ-Iteration Algorithm

Initialize $\overline{\lambda}^0$ and $\underline{\lambda}^0$
Set $t = 0$
repeat
  $\lambda_m^t = \frac{\overline{\lambda}^t + \underline{\lambda}^t}{2}$
  $\hat{P}_i^t = g_i^{-1}(\lambda_m^t) = \frac{\lambda_m^t - b_i}{2a_i}$, $\forall i = 1, \cdots, N$
  $P_i^t = [\hat{P}_i^t]_{P_i^{\min}}^{P_i^{\max}}$, $\forall i = 1, \cdots, N$
  $\Delta P^t = \sum_{i=1}^{N} P_i^t - P_d$
  if $\Delta P^t > 0$ **then**
    $\begin{cases} \overline{\lambda}^t := \lambda_m^t \\ \underline{\lambda}^t := \underline{\lambda}^t \end{cases}$
  else
    $\begin{cases} \overline{\lambda}^t := \overline{\lambda}^t \\ \underline{\lambda}^t := \lambda_m^t \end{cases}$
  end if
  $t = t + 1$
until $\|\overline{\lambda}^t - \underline{\lambda}^t\| < \varepsilon$ is met
output P_i^t

bisection search method, whose pseudocode is summarized in Algorithm 1.

In Algorithm 1, to get a satisfactory optimal solution, it commonly needs dozens of iterations. Furthermore, the computational time would become apparently high when the system scales up. To tackle this challenge, an alternative learning-based approach will be presented in the next section.

III. Learning-Based Approach

In this section, we propose a novel learning-based strategy for solving the ED problem (1) in real time. The core idea of the approach is to take the iterative algorithm as a "learnable" complicated nonlinear mapping, and in this article, to play the role of an excellent "learner," we advocate for the use of DNN.

As the quality of the learned solution inherently depends on the parametrizability to approximate the optimal power allocation function, the algorithm design details are introduced in terms of network structure design, network training and network testing.

A. Network Structure

A fully connected structure consisting of one input layer, one output layer and multiple hidden layers is adopted in this article, as shown in Fig. 2. The total load demand P_d and the optimal ED solution P_i^* are taken as the input and output of this network, respectively. In addition, the rectified linear unit (ReLU) is selected as the activation function in the hidden layers, which is defined as $\mathrm{ReLU}(x) = \max\{0, x\}$. However, it is worth noting that we use a projection operator as a special activation function in the output layer to meet the local generation constraints.

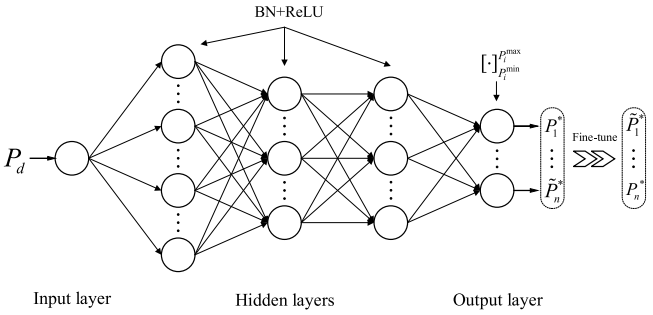

Fig. 2. Fully connected DNN structure used in this article. The output layer use $[\cdot]_{P_i^{\min}}^{P_i^{\max}}$ as the activation to meet the local generation constraints, while the outputs of the DNN is fine-tuned by a simple algorithm to further incorporate the generation-demand balance constraint.

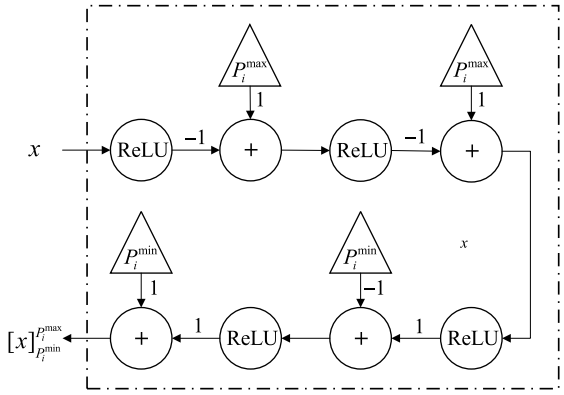

Fig. 3. Implementation of the special activation function $[x]_{P_i^{\min}}^{P_i^{\max}}$.

Specifically, the local constraint projection operator $[x]_{P_i^{\min}}^{P_i^{\max}}$ is defined as

$$[x]_{P_i^{\min}}^{P_i^{\max}} = \begin{cases} P_i^{\max}, & \text{if } x > P_i^{\max} \\ P_i^{\min}, & \text{if } x < P_i^{\min} \\ x, & \text{otherwise.} \end{cases} \tag{5}$$

Such operator can be implemented in the following two successive steps, i.e.,

$$[x]_{P_i^{\min}}^{P_i^{\max}} = P_i^{\min} + \max\Big(\max(v, 0) - P_i^{\min}, 0\Big) \tag{6}$$

where v is the projection operator $[x]_0^{P_i^{\max}}$ and can be constructed with two ReLUs as

$$v = [x]_0^{P_i^{\max}} = P_i^{\max} - \max\big(P_i^{\max} - \max(x, 0), 0\big). \tag{7}$$

The detailed implementation structure of local constraint projection operator is shown in Fig. 3.

Moreover, the output vector of the DNN $[\tilde{P}_1^*, \ldots, \tilde{P}_n^*]^\top$ may be in slightly conflict with the generation-demand balance constraint and local generation constraint so that it requires some fine adjustment. In this article, a simple algorithm proposed in [33] is employed to make the solution obtained by DNN to be feasible, as illustrated below.

First, we detect the average generation-demand mismatch of the whole smart grid system by

$$\delta[n] = \frac{\sum_{i \in DG} \tilde{P}_i^*[n] - P_d}{N} \tag{8}$$

where n denotes the iteration index and $\tilde{P}_i^*[0] = \tilde{P}_i^*$, DG denotes the set of all DGs in a smart grid. Then we update the output variables

$$\tilde{P}_i^*[n+1] = \tilde{P}_i^*[n] + \delta[n] \quad \forall i \tag{9}$$

and conduct the local constraint projection

$$\tilde{P}_i^*[n+1] = \begin{cases} P_i^{\max}, & \text{if } \tilde{P}_i^*[n+1] > P_i^{\max} \\ P_i^{\min}, & \text{if } \tilde{P}_i^*[n+1] < P_i^{\min} \\ \tilde{P}_i^*[n+1], & \text{otherwise.} \end{cases} \tag{10}$$

Let $n = n+1$ and continue the iteration steps in (8)–(10) until the balance between generation and demand is reached, that is $\delta[n] = 0$. If the stopping iteration index is denoted as n_s, then we obtain the final optimal solution

$$\left[P_1^*, \ldots, P_N^*\right]^\top = \left[\tilde{P}_1^*[n_s], \ldots, \tilde{P}_N^*[n_s]\right]^\top. \tag{11}$$

Remark 1: It is worth mentioning that, considering the training computational cost is an important factor to evaluate our approach, we employ the batch normalization (BN) technique in our designed DNN to normalize activation in hidden layers of DNNs and thus accelerate the training process. It has been widely discussed and studied in some works [34]–[36], however, to the best of our knowledge, rarely used in learning-based approaches for power/energy management. Furthermore, since the relationship that DNN needs to express is always considerably complex, the benefits brought by such a technique will be fully revealed.

B. Network Training

Based on the DNN structure shown in Fig. 2, the training data of this approach consists of the input variable P_d and the outputs P_i^*. If these data are available at hand (e.g., history data of a smart grid), they can be used immediately. Otherwise, these training data can be generated by running some existing optimization algorithms, e.g., λ-iteration algorithm, with arbitrary distinct input total load demand since the obtained optimal solutions can be considered as the ground truth. In this way, we can finally get the entire dataset $(P_d^{(k)}\{P_i^{*(k)}\}_{i=1}^N)_{k=1}^K$ where k indexes the data sample. After that, we randomly divide the dataset into a training set, a validation set, and a test set at a ratio of 7:1:2 with reference to commonly used training strategies [37]. Based on the prepared training set, we use the back-propagation technique to continuously optimize the weights of the network so that its outputs gradually approach the ground truth $\{P_i^{*(k)}\}_{i=1}^N$. The loss function provided to the optimizer is the mean square error (MSE) between the ground truth and DNN's output, i.e.,

$$E(\Theta) = \sum_k \sum_{i=1}^N \left\| \hat{P}_i^{*(k)}\left(P_d^{(k)}, \Theta\right) - P_i^{*(k)} \right\|^2 \tag{12}$$

where $\hat{P}_i^{*(k)}(P_d^{(k)}, \Theta)$ is the DNN estimated optimal power output with the network weights Θ and the feeding input $P_d^{(k)}$. The optimizer we employ for such an approximation task is *adam*, which has been widely acclaimed by researchers recently due to its outstanding efficiency and adaptability for a variety of complex optimization problems [38]. Proper learning rate and batchsize will also be chosen by cross-validation. Moreover, to further accelerate the training process, the truncated normal distribution is adopted for the initialization of network weights Θ. The training process stops as long as the validation set's MSE value is lower than a set threshold value.

C. Network Testing

In the testing stage, one of our goals is to verify whether the trained network can handle all possible load demand values. Thus, we feed a large amount of representative P_d from the test set into the network, i.e., with the optimized weights Θ, and collect its outputs $\hat{P}_i^*$. Note that when we split the data set, we have ensured that the samples used for testing must be different from the training samples. Finally, we evaluate the effectiveness of our proposed approach by comparing the MSE and generation cost between the solution obtained by DNN and the ground truth. The MSE is used as an indicator to measure the network approximation accuracy. The generation cost directly reflects the performance of the proposed learning-based approach.

IV. Theoretical Analysis of DNN Approximation

In recent years, some existing studies (e.g., [16] and [39]–[42]) have shown that several specific nonlinear functions can be approximated by DNN. However, it is still not clear whether the existing ED algorithm can be approximated by DNN or not. If the answer is yes, an interesting but challenging question then emerges: What is the quantitative relationship between DNN's structural characteristics (e.g., number of hidden layers and neurons) and its accuracy of approximating an ED algorithm?

In order to answer the above questions, in this section, a rigorous theoretical analysis of the proposed DNN-based approach is presented. Before giving the main results, some useful theoretical support knowledge for the proposed approach is introduced.

A. Preliminaries on Universal Approximation Theory

1) Universal Approximation Theorem for Iterative Algorithm: Let $\mathrm{NET}_N(z)$ be the output of a feedforward neural network consisting of one hidden layer and N sigmoid activation functions with z being the network input. Define x^t as the output of an iterative algorithm at tth iteration, then the relationship between x^t and x^{t+1} is given by

$$x^{t+1} = f^t\left(x^t, z\right), \quad t = 1, \ldots, T \tag{13}$$

where f^t denotes a continuous mapping representing the tth iteration, and T is the total number of iteration.

Lemma 1 [16]: Suppose $x^t \in X, \ \forall t = 1, \ldots, T$ and $z \in Z$ with X and Z being certain compact sets. Then the mapping from the input z to the final output x^T, i.e.,

$$x^T = f^T\left(f^{T-1}\left(\cdots f^1\left(f^0(z), z\right), \ldots, z\right), z\right) \triangleq F(z) \tag{14}$$

can be accurately approximated by $\mathrm{NET}_N(z)$, i.e., for a given approximation error $\varsigma > 0$, there exists a large enough positive

constant N such that

$$\sup_{z\in Z}\|\mathrm{NET}_N(z)-F(z)\|\le\varsigma. \tag{15}$$

The following two lemmas are directly adopted from [16], which are related to the construction of neural networks that can approximate multiplication operation.

Lemma 2 [16]: For any two positive numbers $X_{\max}$, $Y_{\max}$, define

$$\begin{aligned} S &\coloneqq \{(x,y)\mid 0\le x\le X_{\max}, 0\le y\le Y_{\max}\}\\ m &\coloneqq \lceil\log(X_{\max})\rceil \end{aligned} \tag{16}$$

where $\lceil\cdot\rceil$ is a round-up function.

There exists a multilayer neural network with input (x,y) and output $\mathrm{NET}(x,y)$ satisfying the following relation:

$$\max_{(x,y)\in S}|xy-\mathrm{NET}(x,y)|\le\frac{Y_{\max}}{2^n}. \tag{17}$$

To be specific, the polynomial xy can be approximated by $\tilde{x}y$, where $\tilde{x}$ is the n-bit binary expansion of x. In particular

$$\begin{aligned} \tilde{x}y &= \sum_{i=-n}^{m}2^i x_i y\\ &= \sum_{i=-n}^{m}\max\left(2^i y+2^{m+\lceil\log(Y_{\max})\rceil}(x_i-1),0\right) \end{aligned} \tag{18}$$

where x_i is the ith bit of binary expansion of x. In addition, in practice a total of $(2m+2n+3)$ layers, $(m+n+1)$ binary units and $(2m+2n+1)$ ReLUs are needed to complete the approximate multiplication operation.

Lemma 3 [16]: Suppose positive numbers $X_{\max}$, $Y_{\max}$, ϵ_1, ϵ_2 satisfy the following relations:

$$0\le x\le X_{\max},\quad 0\le y\le Y_{\max} \tag{19}$$

$$\max(\epsilon_1,\epsilon_2)\le\max(X_{\max},Y_{\max}),\epsilon_1,\epsilon_2\ge 0. \tag{20}$$

Then, we have

$$\begin{aligned} &|xy-(x\pm\epsilon_1)(y\pm\epsilon_2)|\\ &\quad\le 3\max(X_{\max},Y_{\max})\max(\epsilon_1,\epsilon_2). \end{aligned} \tag{21}$$

B. Main Results

In this section, we are going to provide some theoretical analysis of proposed learning-based approach.

1) Modification of Algorithm 1: It is noted that in Algorithm 1, there exists a noncontinuous operation, i.e., bisection search operation from steps 8 to 12. In order to meet the continuous mapping requirement in Lemma 1, the following sigmoid function is utilized, i.e.,

$$k^t=\mathrm{sigmoid}\left(\Delta P^t\right)=\frac{1}{1+e^{-\alpha\Delta P^t}} \tag{22}$$

where the sigmoid(.) function is used as the fitting function of the step function. An illustration example of sigmoid function with different α values is shown in Fig. 4. It is observed that large enough α can make sigmoid function more fit to the step function.

Then with the help of sigmoid function, Algorithm 1 can be slightly revised, as summarized in Algorithm 2, where $\overline{\lambda}^t$, $\underline{\lambda}^t$ update in Algorithm 1 is changed to the continuous operations shown in steps 8 and 9.

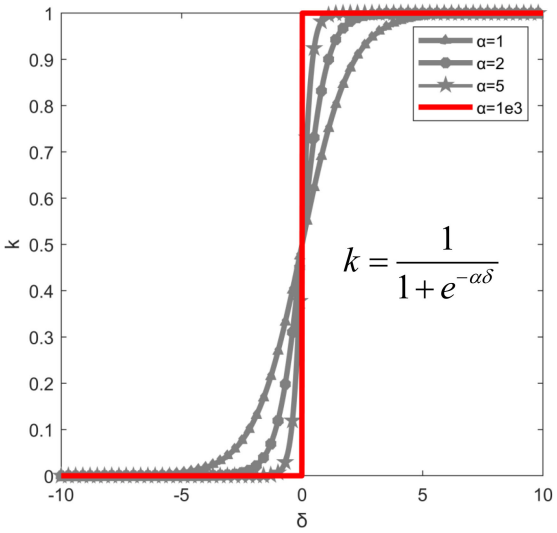

Fig. 4. Illustration of sigmoid function with different α values.

Algorithm 2 Pseudocode of λ-Iteration Algorithm With Sigmoid Function

1: Initialize $\overline{\lambda}^0$ and $\underline{\lambda}^0$
2: Set $t=0$
3: **repeat**
4: $\quad\lambda_m^t=\frac{\overline{\lambda}^t+\underline{\lambda}^t}{2}$
5: $\quad\hat{P}_i^t=g^-(\lambda_m^t)=\frac{\lambda_m^t-b_i}{2a_i},\ \forall i=1,\cdots,N$
6: $\quad P_i^t=[\hat{P}_i^t]_{P_i^{\min}}^{P_i^{\max}},\ \forall i=1,\cdots,N$
7: $\quad\Delta P^t=\sum_{i=1}^{N}P_i^t-P_d$
8: $\quad k^t=\frac{1}{1+e^{-\alpha\Delta P^t}}$
9: $\quad\begin{cases}\overline{\lambda}^{t+1}=\frac{2-k^t}{2}\overline{\lambda}^t+\frac{k^t}{2}\underline{\lambda}^t\\ \underline{\lambda}^{t+1}=\frac{1-k^t}{2}\overline{\lambda}^t+\frac{1+k^t}{2}\underline{\lambda}^t\end{cases}$
10: $\quad t=t+1$
11: **until** $\|\overline{\lambda}^t-\underline{\lambda}^t\|<\varepsilon$ is met
12: **output** P_i^t

Remark 2: Compared to Algorithms 1, Algorithms 2 only differs in the $\overline{\lambda}^t$, $\underline{\lambda}^t$ update, where a sigmoid function is employed. As illustrated in Fig. 4, sufficient large α would make Algorithm 2 almost has no difference from Algorithm 1. However, such modification is essentially useful for the following theoretical analysis of DNN approximation.

2) Main Results: Now we are ready to give the main results as summarized in the following theorem.

Theorem 1: Suppose that λ-iteration algorithm in Algorithm 2 is initialized with $0\le\underline{\lambda}^0\le\overline{\lambda}^0\le\lambda_{\max}$, and stops iteration at $t=T$. Given $\epsilon>0$, there exists a neural network with $P_d\in\mathbb{R}_+$ as input and $\mathrm{NET}(\overline{\lambda}^0,\underline{\lambda}^0,P_d)\in\mathbb{R}_+^N$ as output, with $\mathcal{O}(nT)$ layers, $\mathcal{O}((n+N)T)$ ReLUs, $\mathcal{O}(nT)$ binary units and $\mathcal{O}(T)$ sigmoid units, such that the relation below holds true

$$\max_i\left|P_i^T-\mathrm{NET}\left(\overline{\lambda}^0,\underline{\lambda}^0,P_d\right)_i\right|\le\epsilon \tag{23}$$

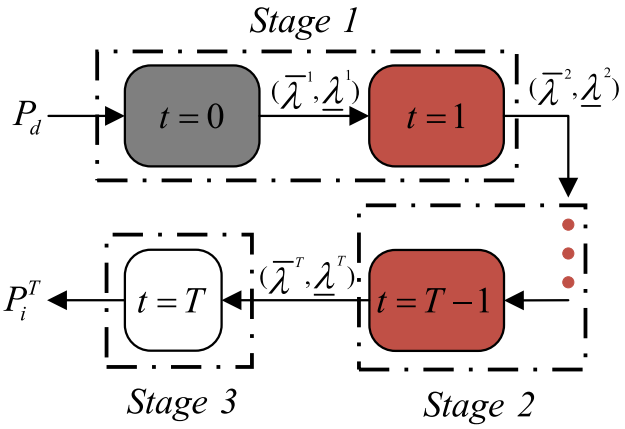

Fig. 5. Illustration of 3-stage approximation analysis of the proposed approach.

where

$$n=\left\lceil -3+\log\frac{\lambda_{\max}Q^{T-2}}{a_{\min}\epsilon}\right\rceil \tag{24}$$

with $a_{\min}$ being the smallest coefficient in $\{a_i\}, i=1,\ldots,N$, and Q being a constant defined as follows:

$$Q\triangleq\frac{3\lambda_{\max}\alpha N}{4a_{\min}}.$$

Proof: Similar to most existing DNN approximation approaches, the main idea of analyzing the DNN approximation accuracy of iterative algorithm is to *deeply unfold* the whole iterative algorithm and then use some basic units to represent the mapping relation in each iteration.

Following this idea, the whole approximation analysis of our proposed approach can be divided into 3 stages, which is illustrated in Fig. 5.

In Stage 1, it mainly involves iterations $t=0$ and $t=1$, where some algorithm initialization operations are conducted and no propagation error exists. During this stage, the mapping $P_d\rightarrow(\overline{\lambda}^2,\underline{\lambda}^2)$ is analyzed in details. While in Stage 2, the mapping from iteration $t=2$ to $t=T-1$, i.e., $(\overline{\lambda}^2,\underline{\lambda}^2)\rightarrow(\overline{\lambda}^T,\underline{\lambda}^T)$ is analyzed. Finally, the whole analysis completes in Stage 3, where the whole propagation error can be obtained by analyzing the mapping $(\overline{\lambda}^T,\underline{\lambda}^T)\rightarrow P_i^T$.

The detailed DNN approximation analysis during each stage can be found as follows.

a) Stage 1: As shown in Fig. 6, at iteration $t=0$, with the prepared initial values $(\overline{\lambda}^0,\underline{\lambda}^0)$ and some system parameters, such as a_i, b_i, we can immediately obtain all intermediate variables before obtaining $(\overline{\lambda}^2,\underline{\lambda}^2)$, including k^1, ΔP^1, $(\overline{\lambda}^1,\underline{\lambda}^1)$, etc, through some simple mathematical manipulations without approximate error. However, when calculating $(\overline{\lambda}^2,\underline{\lambda}^2)$ at iteration $t=1$, we should note that there lacks a direct multiplier operation for two input variables in DNN. Thus, according to the approximation theory in Lemma 2, we can construct a multilayer neural network to approximate such a multiplier operation. In other words, in our constructed DNN, the propagation error is starting from the process of approximately representing $(\overline{\lambda}^2,\underline{\lambda}^2)$.

Denote $\widetilde{k^1}$ as the approximation value of k^1, i.e.,

$$\widetilde{k^1}=k^1-\zeta_{k^1} \tag{25}$$

where $\zeta_{k^1}<2^{-n}$ is the error induced by n-bit binary expansion of k^1, then the approximation errors of $\overline{\lambda}^2$ and $\underline{\lambda}^2$ can be described as follows, respectively

$$\begin{aligned}&\left|\widetilde{\overline{\lambda}^2}-\overline{\lambda}^2\right|\\&=\left|\left(\frac{2-\widetilde{k^1}}{2}\overline{\lambda}^1+\frac{\widetilde{k^1}}{2}\underline{\lambda}^1\right)-\left(\frac{2-k^1}{2}\overline{\lambda}^1+\frac{k^1}{2}\underline{\lambda}^1\right)\right|\\&=\left|\left(\frac{2-k^1+\zeta_{k^1}}{2}\overline{\lambda}^1+\frac{k^1-\zeta_{k^1}}{2}\underline{\lambda}^1\right)-\left(\frac{2-k^1}{2}\overline{\lambda}^1+\frac{k^1}{2}\underline{\lambda}^1\right)\right|\\&=\frac{\overline{\lambda}^1-\underline{\lambda}^1}{2}\zeta_{k^1}=\frac{\overline{\lambda}^0-\underline{\lambda}^0}{4}\zeta_{k^1}\le 2^{-(n+2)}\lambda_{\max}\end{aligned} \tag{26}$$

$$\begin{aligned}&\left|\widetilde{\underline{\lambda}^2}-\underline{\lambda}^2\right|\\&=\left|\left(\frac{1-\widetilde{k^1}}{2}\overline{\lambda}^1+\frac{1+\widetilde{k^1}}{2}\underline{\lambda}^1\right)-\left(\frac{1-k^1}{2}\overline{\lambda}^1+\frac{1+k^1}{2}\underline{\lambda}^1\right)\right|\\&=\frac{\overline{\lambda}^1-\underline{\lambda}^1}{2}\zeta_{k^1}=\frac{\overline{\lambda}^0-\underline{\lambda}^0}{4}\zeta_{k^1}\le 2^{-(n+2)}\lambda_{\max}.\end{aligned} \tag{27}$$

b) Stage 2: In Stage 2, we need to complete the mapping $(\overline{\lambda}^2,\underline{\lambda}^2)\rightarrow(\overline{\lambda}^T,\underline{\lambda}^T)$. The corresponding network architecture for each $t\in[2,T-1]$ is the same as $t=1$ shown in Fig. 6.

First, let us start to analyze by considering the mapping $(\overline{\lambda}^2,\underline{\lambda}^2)\rightarrow(\overline{\lambda}^3,\underline{\lambda}^3)$. Let $\varepsilon=2^{-(n+2)}\lambda_{\max}$, then

$$\begin{aligned}\left|\widetilde{\overline{\lambda}^2}-\overline{\lambda}^2\right|&\le\varepsilon\\\left|\widetilde{\underline{\lambda}^2}-\underline{\lambda}^2\right|&\le\varepsilon\\0<\underline{\lambda}^2&<\overline{\lambda}^2<\lambda_{\max}\end{aligned} \tag{28}$$

which implies that

$$\left|\widetilde{\lambda_m^2}-\lambda_m^2\right|=\left|\frac{\widetilde{\overline{\lambda}^2}+\widetilde{\underline{\lambda}^2}}{2}-\frac{\overline{\lambda}^2+\underline{\lambda}^2}{2}\right|\le\varepsilon$$

and further results in

$$\left|\widetilde{\hat{P}_i^2}-\hat{P}_i^2\right|=\left|\frac{\widetilde{\lambda_m^2}-b_i}{2a_i}-\frac{\lambda_m^2-b_i}{2a_i}\right|\le\frac{\varepsilon}{2a_i}.$$

Since the upper and lower bound projection operators do not magnify the error, therefore

$$\left|\widetilde{P_i^2}-P_i^2\right|\le\left|\widetilde{\hat{P}_i^2}-\hat{P}_i^2\right|\le\frac{\varepsilon}{2a_i}. \tag{29}$$

Then the total power estimation error in step 7 of Algorithm 2 can be obtained as

$$\left|\widetilde{\Delta P^2}-\Delta P^2\right|\le\sum_{i=1}^{N}\frac{\varepsilon}{2a_i}=V \tag{30}$$

which is denoted as V for convenience.

Subsequentially, $|\widetilde{k^2}-k^2|$ can be derived as

$$\begin{aligned}\left|\widetilde{k^2}-k^2\right|&=\left|\frac{1}{1+e^{-\alpha\widetilde{\Delta P^2}}}-\frac{1}{1+e^{-\alpha\Delta P^2}}\right|\\&\le\frac{1}{1+e^{-\alpha\frac{V}{2}}}-\frac{1}{1+e^{\alpha\frac{V}{2}}}=U\end{aligned} \tag{31}$$

which is also denoted as U for convenience.

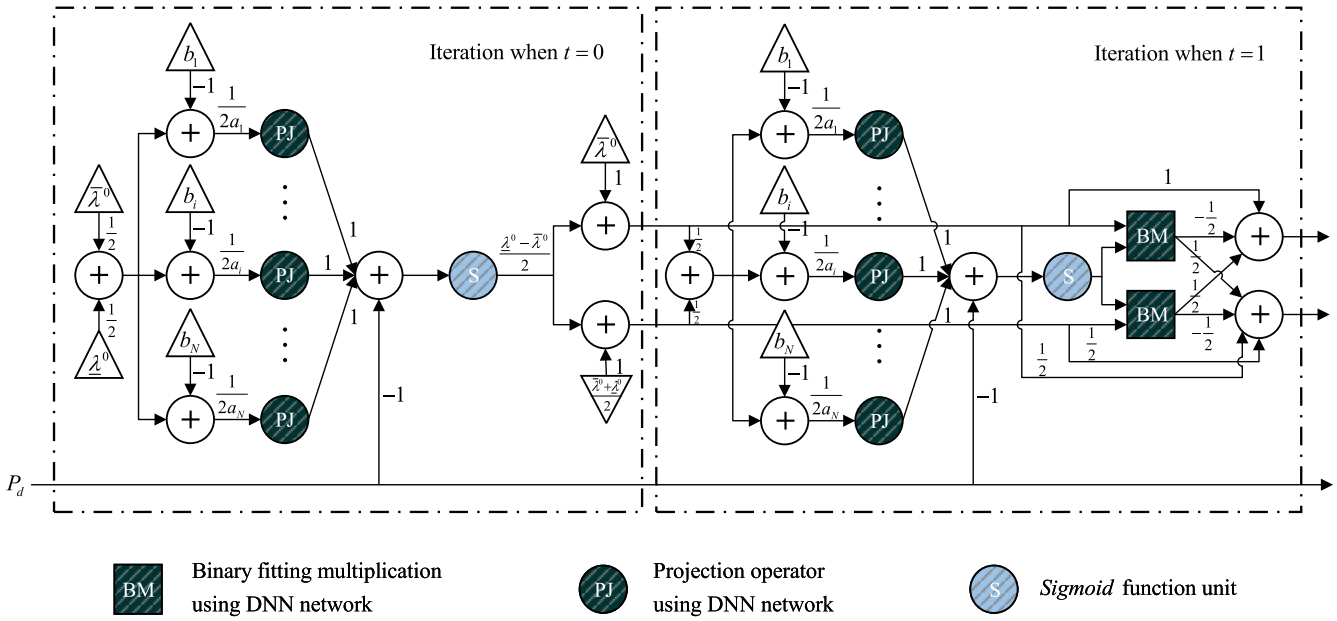

Fig. 6. Consecutive DNN structure in Stage 1.

Now let us consider the approximation error $|\widetilde{\bar{\lambda}^3} - \bar{\lambda}^3|$, i.e.,

$$\left|\widetilde{\bar{\lambda}^3} - \bar{\lambda}^3\right| = \left|\left(\frac{2-\widetilde{\bar{k}^2}}{2}\widetilde{\bar{\lambda}^2} + \frac{\widetilde{\bar{k}^2}}{2}\widetilde{\underline{\lambda}^2}\right) - \left(\frac{2-k^2}{2}\bar{\lambda}^2 + \frac{k^2}{2}\underline{\lambda}^2\right)\right| \le \left|\frac{2-\widetilde{\bar{k}^2}}{2}\widetilde{\bar{\lambda}^2} - \frac{2-k^2}{2}\bar{\lambda}^2\right| + \left|\frac{\widetilde{\bar{k}^2}}{2}\widetilde{\underline{\lambda}^2} - \frac{k^2}{2}\underline{\lambda}^2\right|. \quad (32)$$

For the first term in (32), we have

$$\begin{aligned}&\left|\frac{2-\widetilde{\bar{k}^2}}{2}\widetilde{\bar{\lambda}^2} - \frac{2-k^2}{2}\bar{\lambda}^2\right| \\ &= \left|\frac{2-\left(\widetilde{\bar{k}^2} - \zeta_{\widetilde{\bar{k}^2}}\right)}{2}\widetilde{\bar{\lambda}^2} - \frac{2-k^2}{2}\bar{\lambda}^2\right| \\ &= \left|\left(\frac{2-k^2}{2} + \frac{u+\zeta_{\widetilde{\bar{k}^2}}}{2}\right)\left(\bar{\lambda}^2+\varepsilon_1\right) - \frac{2-k^2}{2}\bar{\lambda}^2\right|\end{aligned}$$

where $\zeta_{\widetilde{\bar{k}^2}} < 2^{-n}$ is the approximation error induced from n-bit binary expansion of $\widetilde{\bar{k}^2}$ in $[(2-\widetilde{\bar{k}^2})/2]\widetilde{\bar{\lambda}^2}$, u is the error between $\widetilde{\bar{k}^2}$ and k^2, ε_1 is the error between $\widetilde{\bar{\lambda}^2}$ and $\bar{\lambda}^2$. Note that $\zeta_{\widetilde{\bar{k}^2}} < 2^{-n}$, $0 < |u| < U$ [i.e., according to (31)] and $0 < |\varepsilon_1| < \varepsilon$ [i.e., according to (28)], then

$$\begin{aligned}&\left|\frac{2-\widetilde{\bar{k}^2}}{2}\widetilde{\bar{\lambda}^2} - \frac{2-k^2}{2}\bar{\lambda}^2\right| \\ &\le \left|\left(\frac{2-k^2}{2} + \frac{U+2^{-n}}{2}\right)\left(\bar{\lambda}^2+\varepsilon\right) - \frac{2-k^2}{2}\bar{\lambda}^2\right| \\ &\le 3\max(1,\lambda_{\max})\max\left(\frac{U+2^{-n}}{2},\varepsilon\right) \quad (33)\end{aligned}$$

where the last inequality is hold due to the results presented in Lemma 3.

Similarly, for the second term in (32), we get

$$\begin{aligned}\left|\frac{\widetilde{\bar{k}^2}}{2}\widetilde{\underline{\lambda}^2} - \frac{k^2}{2}\underline{\lambda}^2\right| &= \left|\frac{\left(\widetilde{\bar{k}^2} - \zeta_{\widetilde{\bar{k}^2}}\right)}{2}\widetilde{\underline{\lambda}^2} - \frac{k^2}{2}\underline{\lambda}^2\right| \\ &= \left|\left(\frac{k^2}{2} + \frac{u-\zeta_{\widetilde{\bar{k}^2}}}{2}\right)\left(\underline{\lambda}^2+\varepsilon_2\right) - \frac{k^2}{2}\underline{\lambda}^2\right| \\ &\le \left|\left(\frac{k^2}{2} + \frac{-U-2^{-n}}{2}\right)\left(\underline{\lambda}^2-\varepsilon\right) - \frac{k^2}{2}\underline{\lambda}^2\right| \\ &\le 3\max\left(\frac{1}{2},\lambda_{\max}\right)\max\left(\frac{U+2^{-n}}{2},\varepsilon\right) \quad (34)\end{aligned}$$

where ε_2 is the error between $\widetilde{\underline{\lambda}^2}$ and $\underline{\lambda}^2$, which also satisfies $0 < |\varepsilon_2| < \varepsilon$.

Substituting (33) and (34) into (32), we have

$$\begin{aligned}\left|\widetilde{\bar{\lambda}^3} - \bar{\lambda}^3\right| &\le 3\max(1,\lambda_{\max})\max\left(\frac{U+2^{-n}}{2},\varepsilon\right) \\ &\quad + 3\max\left(\frac{1}{2},\lambda_{\max}\right)\max\left(\frac{U+2^{-n}}{2},\varepsilon\right). \quad (35)\end{aligned}$$

Similarly, following the analysis steps from (32) to (35), we can get the approximation error $|\widetilde{\underline{\lambda}^3} - \underline{\lambda}^3|$ as:

$$\begin{aligned}&\left|\widetilde{\underline{\lambda}^3} - \underline{\lambda}^3\right| \\ &= \left|\left(\frac{1-\widetilde{\bar{k}^2}}{2}\widetilde{\bar{\lambda}^2} + \frac{1+\widetilde{\bar{k}^2}}{2}\widetilde{\underline{\lambda}^2}\right) - \left(\frac{1-k^2}{2}\bar{\lambda}^2 + \frac{1+k^2}{2}\underline{\lambda}^2\right)\right| \\ &\le \left|\frac{1-\widetilde{\bar{k}^2}}{2}\widetilde{\bar{\lambda}^2} - \frac{1-k^2}{2}\bar{\lambda}^2\right| + \left|\frac{1+\widetilde{\bar{k}^2}}{2}\widetilde{\underline{\lambda}^2} - \frac{1+k^2}{2}\underline{\lambda}^2\right| \\ &\le 3\max\left(\frac{1}{2},\lambda_{\max}\right)\max\left(\frac{U+2^{-n}}{2},\varepsilon\right) \\ &\quad + 3\max(1,\lambda_{\max})\max\left(\frac{U+2^{-n}}{2},\varepsilon\right). \quad (36)\end{aligned}$$

In practice, $\lambda_{\max}$ is usually much larger than 1. In addition, as $V = \sum_{i=1}^{N}(\varepsilon/2a_i)$, which can be further relaxed as

$$\frac{\varepsilon N}{2a_{\max}} \le V \le \frac{\varepsilon N}{2a_{\min}} \tag{37}$$

where $a_{\max} = \max\{a_i\}$ and $a_{\min} = \min\{a_i\}$, $\forall i = 1, \ldots, N$. Consider the following derivations:

$$\begin{aligned} U &\ge \nabla\text{sigmoid}\left(\frac{V}{2}\right)V \\ &\ge \nabla\text{sigmoid}\left(\frac{\varepsilon N}{4a_{\min}}\right)\frac{\varepsilon N}{2a_{\max}} \end{aligned} \tag{38}$$

where ∇ denotes the gradient. Recall the upper bound of ε in (26) and (27), then (38) implies that if α in sigmoid(.) function (22) and n are chosen sufficiently large such that $\nabla\text{sigmoid}(\varepsilon N/4a_{\min}) > (2a_{\max}/N)$, then we readily have $(U/\varepsilon) > 1$. In other words, if $U > \varepsilon$ and $U > 2^{-n}$, then (35) and (36) can be further simplified as

$$\left|\widetilde{\overline{\lambda}}^3 - \overline{\lambda}^3\right| \le 6\lambda_{\max}U \tag{39}$$

$$\left|\widetilde{\underline{\lambda}}^3 - \underline{\lambda}^3\right| \le 6\lambda_{\max}U. \tag{40}$$

Let us further look into (39), we have

$$\left|\widetilde{\overline{\lambda}}^3 - \overline{\lambda}^3\right| \le 6\lambda_{\max}U \le \frac{3\lambda_{\max}\alpha V}{2} \tag{41}$$

where the second inequality is derived from the fact that $(U/V) < \nabla\text{sigmoid}(0)$ with $\nabla\text{sigmoid}(0)$ being the gradient of sigmoid function (22) at point $\Delta P^t = 0$.

Combining (37) and (41) yields

$$\left|\widetilde{\overline{\lambda}}^3 - \overline{\lambda}^3\right| \le \frac{3\lambda_{\max}\alpha N}{4a_{\min}}\varepsilon = Q\varepsilon \tag{42}$$

where Q is given below

$$Q \triangleq \frac{3\lambda_{\max}\alpha N}{4a_{\min}}. \tag{43}$$

Similarly, (40) can be also relaxed as

$$\left|\widetilde{\underline{\lambda}}^3 - \underline{\lambda}^3\right| \le Q\varepsilon. \tag{44}$$

By comparing the relations in (28), (42), and (44), it can be concluded that the approximation errors of neural network between iterations $t = 2$ and $t = 3$ is bounded, specifically,

$$\begin{aligned} \left|\widetilde{\overline{\lambda}}^3 - \overline{\lambda}^3\right| &\le Q\left|\widetilde{\overline{\lambda}}^2 - \overline{\lambda}^2\right| \\ \left|\widetilde{\underline{\lambda}}^3 - \underline{\lambda}^3\right| &\le Q\left|\widetilde{\underline{\lambda}}^2 - \underline{\lambda}^2\right|. \end{aligned} \tag{45}$$

By repeatedly analyzing the approximation error propagation from (28) to (45), the approximation errors for the mapping $(\overline{\lambda}^2, \underline{\lambda}^2) \to (\overline{\lambda}^T, \underline{\lambda}^T)$ can be quantified as

$$\begin{aligned} \left|\widetilde{\overline{\lambda}}^T - \overline{\lambda}^T\right| &\le Q^{T-2}\left|\widetilde{\overline{\lambda}}^2 - \overline{\lambda}^2\right| \le Q^{T-2}\varepsilon \\ \left|\widetilde{\underline{\lambda}}^T - \underline{\lambda}^T\right| &\le Q^{T-2}\left|\widetilde{\underline{\lambda}}^2 - \underline{\lambda}^2\right| \le Q^{T-2}\varepsilon. \end{aligned} \tag{46}$$

c) Stage 3: In this stage, we are approaching to calculate the final approximation error. Following the analysis steps from (28) to (29), the approximation error of the optimal power output for the ith generator after T iteration can be immediately obtained from (46) as:

$$\left|\widetilde{P_i^T} - P_i^T\right| \le \frac{1}{2a_i}Q^{T-2}\varepsilon \tag{47}$$

which further indicates that

$$\max_i \left|P_i^T - \text{NET}\left(\overline{\lambda}^0, \underline{\lambda}^0, P_d\right)_i\right| \le \frac{1}{2a_{\min}}Q^{T-2}\varepsilon. \tag{48}$$

Suppose the approximation error for the constructed DNN is bounded by $\epsilon > 0$, i.e., the result in (23) is hold, then

$$2^{-(n+3)}\frac{\lambda_{\max}Q^{T-2}}{a_{\min}} \le \epsilon \tag{49}$$

which implies the following requirement for the bits of binary expansion:

$$n \ge -3 + \log\frac{\lambda_{\max}Q^{T-2}}{a_{\min}\epsilon} \tag{50}$$

where log(.) is logarithm with base 2.

Therefore, in order to ensure (23) in Theorem 1 holds, the minimum bits of neural network binary expansion can be selected as

$$\left\lceil -3 + \log\frac{\lambda_{\max}Q^{T-2}}{a_{\min}\epsilon}\right\rceil.$$

In addition, through above analysis, we can easily count the number of neurons and layers we used to approximate the whole algorithm. In Stage 1, when $t = 0$ we need 4 layers, $4N$ ReLUs, 1 sigmoid unit; when $t = 1$, $(2n + 7)$ layers, $(4N + 4n + 2)$ ReLUs, 1 sigmoid unit and $(2n + 2)$ binary units are needed. In Stage 2, layers and units required in each iteration are the same as those in $t = 1$ of Stage 1. In Stage 3, 4 layers, $4N$ ReLUs are required.

In all, we need at most $[(T-1)(2n+7)+8]$ layers, $[(T-1)(4N+4n+2)+8N]$ ReLUs, $[(T-1)(2n+2)]$ binary units and T sigmoid units.

This completes the proof. ■

Remark 3: It is worth pointing out that based on our practical observation, the size of the neural network (i.e., the numbers of layers, ReLUs, binary units and sigmoid units) predicted by Theorem 1 would be much smaller when achieving similar satisfactory approximation accuracy. This remark will be verified by the combined results of case study 1 and case study 2 in the next section.

V. Case Studies

In this section, in order to validate the effectiveness of proposed approach, two ED case studies are carried out. In the first case, a small-scale smart grid system consisting of 3 generators is considered, where a DNN is strictly constructed according to the theoretical results derived in Theorem 1 to check its approximation accuracy. In case study 2, we test our proposed approach in an IEEE 30-bus power system, where a DNN only with three hidden layers is employed to learn the λ-iteration algorithm.

TABLE I
COST FUNCTION PARAMETERS OF GENERATORS IN CASE STUDY 1

DG	a_i $[\$/MW^2h]$	b_i $[\$/MWh]$	c_i $[\$/h]$	$P_i^{\min}$ $[MW]$	$P_i^{\max}$ $[MW]$
1	0.0025	1	170	10	100
2	0.004	1.6	170	20	100
3	0.005	1.25	200	10	120

TABLE II
PARAMETERS OF ALGORITHM 2 IN CASE STUDY 1

Concept	Symbol	Value
Upper bound of λ	$\lambda_{\max}$	4e3
Initial value of $\overline{\lambda}$	$\overline{\lambda}^0$	4e3
Initial value of $\underline{\lambda}$	$\underline{\lambda}^0$	0
Coefficient of sigmoid	α	1e3
Number of generators	N	3
Smallest coefficient of a	$a_{\min}$	0.0025
Total Iteration	T	28
Given approximation error P_i^T	ϵ	1e-3

The code generating the simulation results in this article is available online: https://github.com/xbwwwx/DNN-ED-IoT.

A. Case Study 1

In this section, a smart grid system with 3 DGs is considered to verify our theoretical findings established in Theorem 1. The parameters of DGs' cost function are listed in Table I.

The modified λ-iteration algorithm, i.e., Algorithm 2, is used as a base algorithm, which in fact acts as the mapping $(\overline{\lambda}^0, \underline{\lambda}^0, P_d) \rightarrow P_i$. Following the idea of "deeply unfolding" Algorithm 2, we construct a DNN that strictly "mimic" Algorithm 2. The detailed constructed DNN structure is shown in Fig. 6 and its corresponding key parameters are summarized in Table II.

First, we test our constructed DNN performance by fixing the total load demand as $P_d = 100$ MW, which is directly fed into our constructed DNN. The power outputs of three DGs at each iteration are shown in Fig. 7. It is observed that our constructed DNN can exactly follow the outputs of Algorithm 2 in each iteration step. Then we also test the fitting performance of the constructed DNN with different load demand P_d. In this case, P_d is designed to vary in the range of [50 MW, 200 MW] and we compare the output at iteration $t = 28$. Based on the results in Theorem 1, we construct a DNN with 46 256 layers, 92 526 ReLUs, 46 116 binary units, and 28 sigmoid units, where the minimum bits of binary expansion n is chosen as $n = 853$. The comparison results are shown in Fig. 8. It can be observed that the constructed DNN can perfectly approximate Algorithm 2 with extremely high accuracy, where the approximation error can reach to a level of $e-13$. These results verify our theoretical findings in Theorem 1. However, as pointed out in Remark 2, in practice a DNN with much smaller size can also achieve good approximation accuracy, which is discussed in the next case study.

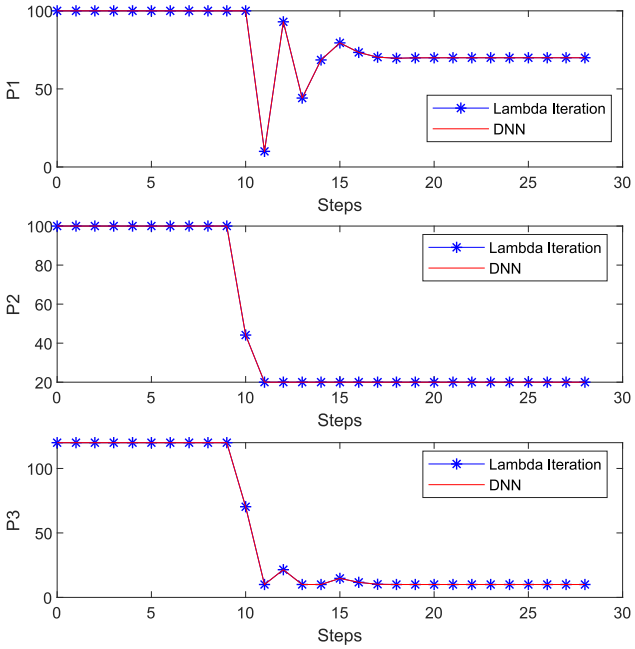

Fig. 7. Approximation performance of constructed DNN with fixed load demand $P_d = 100$ MW.

TABLE III
GENERATOR PARAMETERS IN IEEE 30-BUS POWER SYSTEM

DG	α_i $[\$/MW^2h]$	β_i $[\$/MWh]$	γ_i $[\$/h]$	$P_i^{\min}$ $[MW]$	$P_i^{\max}$ $[MW]$
1	0.00375	2	0	50	200
2	0.0175	1.75	0	20	80
3	0.0625	1.0	0	15	50
4	0.00834	3.25	0	10	35
5	0.025	3.0	0	10	30
6	0.025	3.0	0	12	40

B. Case Study 2

1) IEEE 30-Bus Test System: In order to test the effectiveness of proposed real-time ED algorithm, the IEEE 30-bus system containing 6 DGs is chosen as the test system in this section. The generator parameters listed in Table III are taken from [10].

2) Experimental Setup: In this case study, P_d is assumed to vary from 141.7 MW to 425.1 MW which covers 50%–150% of its nominal value 283.4 MW. Such an interval is considered as a feasible interval for P_d in this case, and thus the dataset sufficiently sampled from this interval is considered representative. Different from that in case study 1, in this case study we build a DNN with three hidden layers while each one contains 200 neurons. The training process stops when the MSE value of the validation set is less than 1e-3.

In order to get a faster and stable training process, we use cross-validation to find the optimal training hyperparameters. The impact of batch size and learning rate on the early training performance in terms of the MSE value of the validation set is shown in Fig. 9. We can observe that the convergence rate is slowed down when the batch size is too large, while the unstable convergence behavior occurs when the batch size is

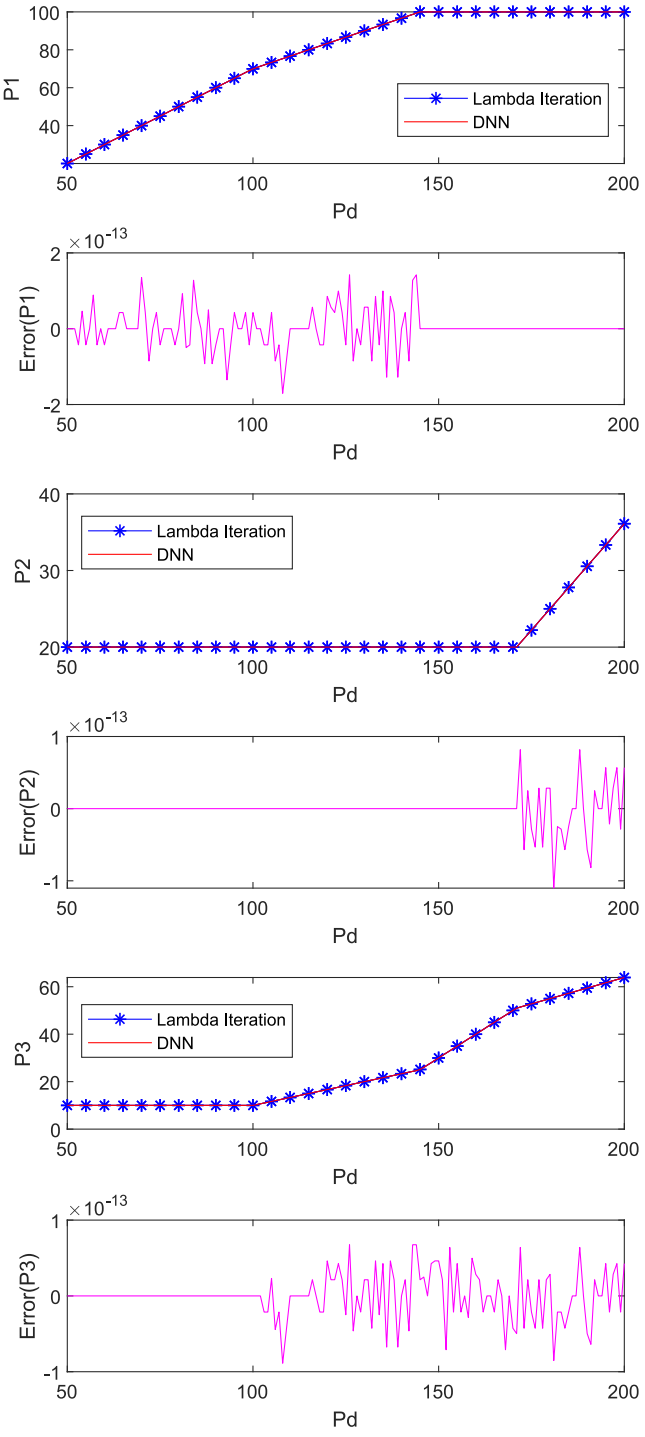

Fig. 8. Approximation performance of constructed DNN with varying load demand P_d in case study 1.

too small. Besides, we can also observe that the convergence trend of network training appears within tens of seconds when using different learning rates. Combining the above experimental results and inherent experience, the batch size and learning rate are selected as 1000 and 0.001, respectively, in this study.

The training tasks involved in this article are all completed on a server equipped with four Intel i7-6950X 3.0-GHz CPUs, two Nvidia K40 GPUs, and 128 GB of memory.

3) Performance: As discussed in Remark 1, BN makes a significant contribution to improving the training efficiency. Therefore, we plot the least MSE curve of the validation set to compare the training efficiency of DNN with and without BN, as shown in Fig. 10. It can be observed that the training of DNN with BN is completed at 270545th epoch while the one without BN distinctly suffers from a slow convergence rate as the least MSE of the validation set is still 0.335 at 1e6th epoch.

Once our proposed DNN is ready, i.e., well trained by using above training data set and passed the cross validation, we begin to test its performance by feeding P_d in the test data set and then comparing the outputs produced by this DNN and the targets in the test data set. The comparison results are shown in Fig. 11. We can observe that although only three hidden layers are used in this DNN network, it can still achieve high approximation accuracy. The total MSE of the whole test data between our proposed approach and Algorithm 1 is 1.01e-3. Besides, the computational time of our DNN-based approach is calculated as 4.63e-4 s, which is far more shorter than most of existing ED optimization algorithms. These results indicate that our approach provides an alternative way to realize real-time ED optimization in future smart grid system.

C. Case Study 3: Dynamic Units Status

Considering that in practical scenarios, some generators will be turned on/off for specific reasons, therefore, our designed algorithm should be able to adapt to different statuses of generators. Motivated by this consideration, the generators status $[U_N, \ldots, U_1]$, in addition to the total load demand P_d, is also taken as the input of DNN in this case study, where $U_i = 1$ indicates the ith generator is on, otherwise it is off.

The IEEE 30-bus power system is also adopted here as the test system and the generation constraints are also considered. To build a sufficiently representative dataset, the units status$[U_N, \ldots, U_1]$ traverses all feasible generators status and P_d is uniformly sampled in the corresponding feasible region. The dataset generated for this study consists of 1017663 samples. Since the relationship to be expressed by DNN is far more complex than the one of case study 2, the DNN used in this study contains ten hidden layers. The rest of the experimental settings are the same as those in Section V-B2.

As shown in Fig. 12, there are 3000 test samples visualized in the 3-D space where each point represents an optimal solution obtained by the λ-iteration algorithm. In Fig. 12(a) and (b), the color of each point denotes the corresponding MSE and the difference in cost between the solutions obtained by the λ-iteration algorithm and DNN, respectively. It can be observed that our proposed approach generates satisfactory solutions with the average MSE being 1.02e-3 and the average cost error being 0.125 $/h (the average percentage difference is 0.026%). Besides, the DNN only takes an average time of 1.21 ms, which indicates that it achieves excellent real-time capability to meet the high requirements of dynamic ED.

D. Case Study 4: Different Frontal Algorithms

To study the impact of different frontal algorithms (i.e., the algorithm used to obtain the ground truth) on the trained

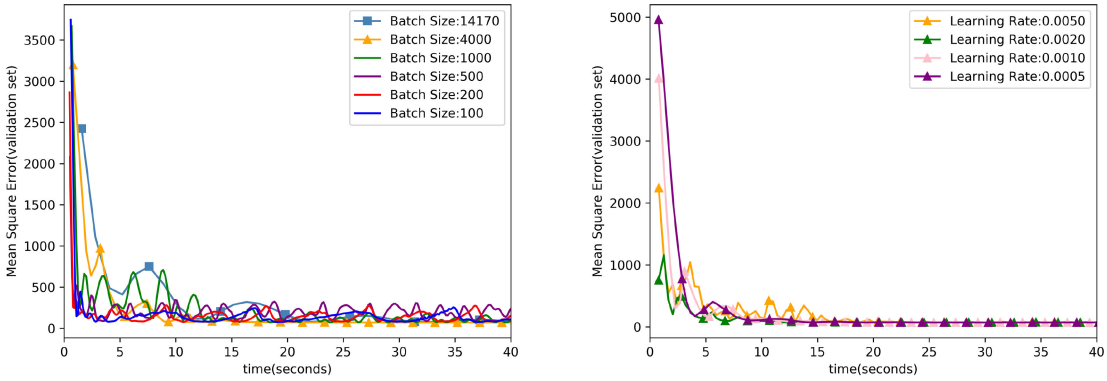

Fig. 9. DNN hyperparameter selection for the IEEE-30 bus power system in case study 2. (a) Batch size selection. (b) Learning rate selection.

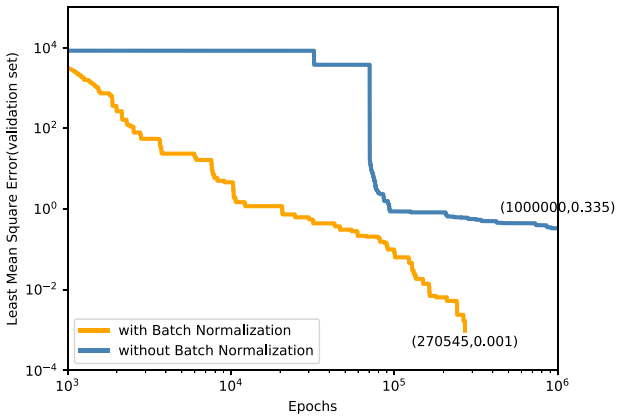

Fig. 10. Convergence performance of DNNs trained with and without BN. The training process stops when the MSE value of the validation set is less than 1e-3.

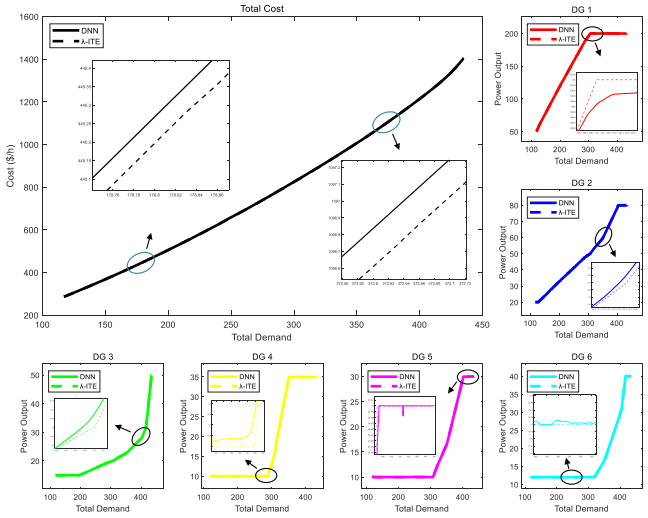

Fig. 11. Trained DNN approximation/cost performance visualized by 6360 test samples with average MSE being 9.92e-4 in case study 2.

DNN's performance, in addition to the λ-iteration algorithm, another convex optimization algorithm named HDO [10] is implemented to obtain the ground truth, with which another DNN is trained. In this case, the total load demand is set

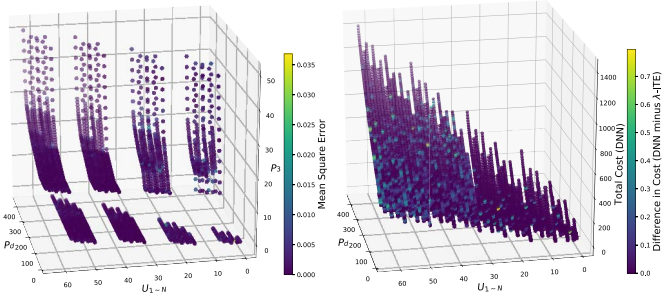

Fig. 12. In case study 3, we randomly select 3000 test samples from the test set to show the performance of the DNN. $U_{1\sim N}$ is the decimal variable converted from the set of the binary variables $[U_N, \ldots, U_1]$ for easy visualization. (a) Trained DNN approximation performance with the average MSE being 1.02e-3. (b) Trained DNN cost performance with the average cost error being 0.125 $/h (the average percentage difference is 0.026%).

TABLE IV
COMPARISON OF DIFFERENT FRONTAL ALGORITHMS

Power Allocation (MW)	λ-Ite.	DNN (λ-Ite.)	HDO	DNN (HDO)
P_1	185.4035	185.3728	185.4029	185.3704
P_2	46.8722	46.8926	46.8724	46.8594
P_3	19.1242	19.1089	19.1244	19.1703
P_4	10.0000	10.0000	10.0001	10.0000
P_5	10.0000	10.0257	10.0001	10.0000
P_6	12.0000	12.0000	12.0001	12.0000
Total Generation Cost ($/h)	767.6018	767.6050	767.6021	767.6026

to $P_d = 283.4$ MW. The comparison results are shown in Table IV. We can observe that the solutions obtained by the λ-iteration algorithm and the HDO algorithm are very similar. It is not a surprising result since the formulated problem is a convex optimization problem, whose optimal solution can be obtained by both the λ-iteration and HDO algorithms. Based on this, we can easily find that the performances of the two DNNs, respectively, using these two algorithms as the frontal algorithm are similar.

VI. CONCLUSION

Motivated by recent progress in machine learning technology, in this article, we present a novel learning-based optimization algorithm to address the ED problem in a smart

grid system. Different from most of existing numerical ED optimization algorithms, no computational-costly mathematical calculation is required in this approach, instead only some simple matrix-vector multiplication in neural network is conducted, which makes it suitable for on-line optimization. In addition, to explain why such DNN can well approximate the iterative optimization algorithm, a detailed theoretical analysis of DNN universal approximation is provided, which indicates that a modified λ-iteration algorithm can be well approximated by a finite-size DNN. However, it has been also revealed that there exists a gap between such theoretical analysis and practical implementation, i.e., a DNN with much smaller network size can also "learn" the optimization algorithm with satisfactory accuracy.

Our future work includes developing distributed variants of this method and considering more complex ED models. In addition, using our "learning to optimize" idea to solve other interesting but challenging problems in smart grids and other areas, such as unit commitment, energy storage optimization, and robust optimization [43], [44], will be an important direction of our future work.

REFERENCES

[1] X. Yu and Y. Xue, "Smart grids: A cyber–physical systems perspective," *Proc. IEEE*, vol. 104, no. 5, pp. 1058–1070, May 2016.

[2] F. Guo, C. Wen, and Y.-D. Song, *Distributed Control and Optimization Technologies in Smart Grid Systems*. New York, NY, USA: CRC Press, 2017.

[3] A. Askarzadeh, "A memory-based genetic algorithm for optimization of power generation in a microgrid," *IEEE Trans. Sustain. Energy*, vol. 9, no. 3, pp. 1081–1089, Jul. 2018.

[4] Q. Liu, X. Le, and K. Li, "A distributed optimization algorithm based on multiagent network for economic dispatch with region partitioning," *IEEE Trans. Cybern.*, vol. 51, no. 5, pp. 2466–2475, May 2021, doi: 10.1109/TCYB.2019.2948424.

[5] F. Guo, C. Wen, J. Mao, and Y.-D. Song, "Distributed economic dispatch for smart grids with random wind power," *IEEE Trans. Smart Grid*, vol. 7, no. 3, pp. 1572–1583, May 2016.

[6] C. Li, X. Yu, W. Yu, T. Huang, and Z. Liu, "Distributed event-triggered scheme for economic dispatch in smart grids," *IEEE Trans. Ind. Informat.*, vol. 12, no. 5, pp. 1775–1785, Oct. 2016.

[7] F. Guo, G. Li, C. Wen, L. Wang, and Z. Meng, "An accelerated distributed gradient-based algorithm for constrained optimization with application to economic dispatch in a large-scale power system," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 51, no. 4, Apr. 2021, pp. 2041–2053, doi: 10.1109/TSMC.2019.2936829.

[8] G. Chen and Q. Yang, "An ADMM-based distributed algorithm for economic dispatch in islanded microgrids," *IEEE Trans. Ind. Informat.*, vol. 14, no. 9, pp. 3892–3903, Sep. 2018.

[9] Y. Zheng, Y. Song, D. J. Hill, and Y. Zhang, "Multiagent system based microgrid energy management via asynchronous consensus ADMM," *IEEE Trans. Energy Convers.*, vol. 33, no. 2, pp. 886–888, Jun. 2018.

[10] F. Guo, C. Wen, J. Mao, J. Chen, and Y. Song, "Hierarchical decentralized optimization architecture for economic dispatch: A new approach for large-scale power system," *IEEE Trans. Ind. Informat.*, vol. 14, no. 2, pp. 523–534, Feb. 2018.

[11] C.-L. Chiang, "Improved genetic algorithm for power economic dispatch of units with valve-point effects and multiple fuels," *IEEE Trans. Power Syst.*, vol. 20, no. 4, pp. 1690–1699, Nov. 2005.

[12] M. Nemati, M. Braun, and S. Tenbohlen, "Optimization of unit commitment and economic dispatch in microgrids based on genetic algorithm and mixed integer linear programming," *Appl. Energy*, vol. 210, pp. 944–963, Jan. 2018.

[13] A. Meng, J. Li, and H. Yin, "An efficient crisscross optimization solution to large-scale non-convex economic load dispatch with multiple fuel types and valve-point effects," *Energy*, vol. 113, pp. 1147–1161, Oct. 2016.

[14] W. T. Elsayed, Y. G. Hegazy, M. S. El-bages, and F. M. Bendary, "Improved random drift particle swarm optimization with self-adaptive mechanism for solving the power economic dispatch problem," *IEEE Trans. Ind. Informat.*, vol. 13, no. 3, pp. 1017–1026, Jun. 2017.

[15] N. Noman and H. Iba, "Differential evolution for economic load dispatch problems," *Electr. Power Syst. Res.*, vol. 78, no. 8, pp. 1322–1331, 2008.

[16] H. Sun, X. Chen, Q. Shi, M. Hong, X. Fu, and N. D. Sidiropoulos, "Learning to optimize: Training deep neural networks for interference management," *IEEE Trans. Signal Process.*, vol. 66, no. 20, pp. 5438–5453, Oct. 2018.

[17] T. Fu, C. Wang, and N. Cheng, "Deep learning based joint optimization of renewable energy storage and routing in vehicular energy network," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6229–6241, Jul. 2020.

[18] K. Li and J. Malik, "Learning to optimize," 2016. [Online]. Available: arXiv:1606.01885.

[19] H. Yang, W.-D. Zhong, C. Chen, A. Alphones, and X. Xie, "Deep reinforcement learning based energy-efficient resource management for social and cognitive Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5677–5689, Jun. 2020.

[20] C. Zheng, Z. Li, Y. Yang, and S. Wu, "Exposure interpolation via fusing conventional and deep learning methods," 2019. [Online]. Available: arXiv:1905.03890.

[21] K.-A. Toh, Z. Lin, Z. Li, B. Oh, and L. Sun, "Gradient-free learning based on the kernel and the range space," 2018. [Online]. Available: arXiv:1810.11581.

[22] Z. Ling, X. Tao, Y. Zhang, and X. Chen, "Solving optimization problems through fully convolutional networks: An application to the traveling salesman problem," *IEEE Trans. Syst., Man, Cybern., Syst.*, early access, Feb. 11, 2020, doi: 10.1109/TSMC.2020.2969317.

[23] F. Li, J. Qin, and W. X. Zheng, "Distributed Q-learning-based online optimization algorithm for unit commitment and dispatch in smart grid," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 4146–4156, Sep. 2020, doi: 10.1109/TCYB.2019.2921475.

[24] L. Lin, X. Guan, Y. Peng, N. Wang, S. Maharjan, and T. Ohtsuki, "Deep reinforcement learning for economic dispatch of virtual power plant in Internet of Energy," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6288–6301, Jul. 2020.

[25] Y. Yang, Z. Yang, J. Yu, K. Xie, and L. Jin, "Fast economic dispatch in smart grids using deep learning: An active constraint screening approach," *IEEE Internet Things J.*, vol. 7, no. 11, pp. 11030–11040, Nov. 2020.

[26] J. Nanda *et al.*, "Application of artificial neural network to economic load dispatch," in *Proc. 4th Int. Conf. Adv. Power Syst. Control, Oper. Manage. (APSCOM)*, vol. 2, Hong Kong, 1997, pp. 707–711, doi: 10.1049/cp:19971920.

[27] S. Panta, S. Premrudeepreechachani, S. Nuchprayoon, C. Dechthummarong, S. Janjornmanit, and S. Yachiangkam, "Optimal economic dispatch for power generation using artificial neural network," in *Proc. Int. Power Eng. Conf. (IPEC)*, Dec. 2007, pp. 1343–1348.

[28] T. Salimans and D. P. Kingma, "Weight normalization: A simple reparameterization to accelerate training of deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 901–909.

[29] J. Alsumait, J. Sykulski, and A. Al-Othman, "A hybrid GA–PS–SQP method to solve power system valve-point economic dispatch problems," *Appl. Energy*, vol. 87, no. 5, pp. 1773–1781, 2010.

[30] K. P. Wong and Y. W. Wong, "Hybrid genetic/simulated annealing approach to short-term multiple-fuel-constrained generation scheduling," *IEEE Trans. Power Syst.*, vol. 12, no. 2, pp. 776–784, May 1997.

[31] A. J. Wood, B. F. Wollenberg, and G. B. Sheblé, *Power Generation, Operation, and Control*. Hoboken, NJ, USA: Wiley, 2013.

[32] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[33] F. Li, J. Qin, Y. Kang, and W. X. Zheng, "Consensus based distributed reinforcement learning for nonconvex economic power dispatch in microgrids," in *Proc. Int. Conf. Neural Inf. Process.*, 2017, pp. 831–839.

[34] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," 2015. [Online]. Available: arXiv:1502.03167.

[35] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, "How does batch normalization help optimization?" in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 2483–2493.

[36] N. Bjorck, C. P. Gomes, B. Selman, and K. Q. Weinberger, "Understanding batch normalization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 7694–7705.

[37] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014. [Online]. Available: arXiv:1412.6980.
[39] S. Liang and R. Srikant, "Why deep neural networks for function approximation?" 2016. [Online]. Available: arXiv:1610.04161.
[40] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.
[41] X. Le, S. Chen, Z. Yan, and J. Xi, "A neurodynamic approach to distributed optimization with globally coupled constraints," *IEEE Trans. Cybern.*, vol. 48, no. 11, pp. 3149–3158, Nov. 2018.
[42] F. Liang, C. Shen, W. Yu, and F. Wu, "Towards optimal power control via ensembling deep neural networks," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1760–1776, Mar. 2020.
[43] Y. Liu, L. Wu, and J. Li, "A fast LP-based approach for robust dynamic economic dispatch problem: A feasible region projection method," *IEEE Trans. Power Syst.*, vol. 35, no. 5, pp. 4116–4119, Sep. 2020.
[44] A. Lorca and X. A. Sun, "Adaptive robust optimization with dynamic uncertainty sets for multi-period economic dispatch under significant wind," *IEEE Trans. Power Syst.*, vol. 30, no. 4, pp. 1702–1713, Jul. 2015.

Fanghong Guo (Member, IEEE) received the B.Eng. degree in automation from Southeast University, Nanjing, China, in July 2010, the M.Eng. degree in automation science and electrical engineering from Beihang University, Beijing, China, in January 2013, and the Ph.D. degree in sustainable earth from Energy Research Institute @NTU, Interdisciplinary Graduate School, Nanyang Technological University, Singapore, in November 2016.

He was a Research Associate and then a Research Fellow with Rolls-Royce Cooperate Lab @NTU, Nanyang Technological University, from May 2016 to April 2017. From May 2017 to July 2018, he was a Scientist with Experimental Power Grid Centre, Agency for Science, Technology and Research (A*STAR), Singapore. He is currently with the Department of Automation, Zhejiang University of Technology, Hangzhou, China. His research interests include distributed cooperative control, distributed optimization on microgrid systems, and smart grid.

Bowen Xu received the B.Eng. degree in communication engineering from Zhejiang University of Technology, Hangzhou, China, in 2019, where he is currently pursuing the M.S. degree with the Department of Automation.

His research interests include optimization, deep learning, and its applications in smart grids.

Lantao Xing (Member, IEEE) received the B.Eng degree in automation from China University of Petroleum (East China), Dongying, China, in 2013, and the Ph.D. degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2018.

He worked as a Reasearch Fellow with the School of Computer Science, Queensland University of Technology, Brisbane, QLD, Australia. He is currently a Presidential Postdoctoral Fellow with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. His research interests include nonlinear system control, event-triggered control, and distributed control with applications in smart grid.

Wen-An Zhang (Member, IEEE) received the B.Eng. degree in automation and the Ph.D. degree in control theory and control engineering from the Zhejiang University of Technology, Hangzhou, China, in 2004 and 2010, respectively.

He has been with the Zhejiang University of Technology since 2010 where he is currently a Professor with the Department of Automation. He was a Senior Research Associate with the Department of Manufacturing Engineering and Engineering Management, City University of Hong Kong, Hong Kong, from 2010 to 2011. His current research interests include networked control systems, multisensor information fusion estimation, and robotics.

Prof. Zhang was awarded an Alexander von Humboldt Fellowship from 2011 to 2012. He has been serving as a Subject Editor for *Optimal Control Applications and Methods* from September 2016.

Changyun Wen (Fellow, IEEE) received the B.Eng. degree from Xi'an Jiaotong University, Xi'an, China, in 1983, and the Ph.D. degree from the University of Newcastle, Callaghan, NSW, Australia, in 1990.

From August 1989 to August 1991, he was a Postdoctoral Fellow with the University of Adelaide, Adelaide, SA, Australia. Since August 1991, he has been with Nanyang Technological University, Singapore, where he is currently a Full Professor. His main research activities are in the areas of control systems and applications, cyber–physical systems, smart grids, complex systems, and networks. Some of his publications in these areas are available in https://publons.com/researcher/2816818/changyun-wen/publications/.

Prof. Wen was a recipient of the number of awards, including the Prestigious Engineering Achievement Award from the Institution of Engineers, Singapore, in 2005, and the Best Paper Award of IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS in 2017. He is currently the Co-Editor-in-Chief of IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, an Associate Editor of *Automatica* (from February 2006), and the Executive Editor-in-Chief of *Journal of Control and Decision*. He also served as an Associate Editor for IEEE TRANSACTIONS ON AUTOMATIC CONTROL from 2000 to 2002, IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS from 2013 to 2019, and *IEEE Control Systems Magazine* from 2009 to 2019. He has been actively involved in organizing international conferences playing the roles of General Chair (including the General Chair of IECON 2020 and IECON 2023) and the TPC Chair (e.g., the TPC Chair of Chinese Control and Decision Conference since 2008). He was a member of IEEE Fellow Committee from January 2011 to December 2013 and a Distinguished Lecturer of IEEE Control Systems Society from 2010 to 2013.

Li Yu (Senior Member, IEEE) received the B.S. degree from the Nankai University, Tianjin, China, in 1985, and the M.S. and Ph.D. degrees in control system and engineering from Zhejiang University, Hangzhou, China, in 1988 and 1999, respectively.

He is currently a Professor in Automation with the College of Information Engineering, Zhejiang University of Technology, Hangzhou. He has authored or coauthored three books and over 200 journal or conference papers. His current research interests include robust control, time-delay systems, networked control systems, and motion control.